

SERVICIO NACIONAL DE APRENDIZAJE – SENA

Centro de Servicios y Gestión Empresarial · Coordinación de Teleinformática, SENA Regional Antioquia

MATRIZ DE VALIDACIÓN TÉCNICA – PROYECTO INTEGRADOR REACT + FASTAPI

Ficha:	3406211	Programa:	Tecnólogo en Análisis y Desarrollo de Software (código 228118)
Aprendiz:	JUAN DIEGO CARTAGENA TUBERQUIA	Instructor:	César Augusto Moreno Mena
Proyecto:	Catálogo de Motos – BIXE	Fecha de validación:	_____

Competencia (Norma SofíaPlus)	Resultado de Aprendizaje (RAP)
38367 - Estructurar propuesta técnica de servicio de TI según requisitos técnicos y normativa	593060 - 01 Definir especificaciones técnicas del software de acuerdo con las características del software a construir.
38356 - Implementar la solución de software de acuerdo con los requisitos de operación y modelos de referencia	593109 - 03 Documentar el proceso de implantación de software siguiendo estándares de calidad.
38356 - Implementar la solución de software de acuerdo con los requisitos de operación y modelos de referencia	593112 - 04 Implantar el software de acuerdo con los niveles de servicio establecidos con el cliente.

Criterio verificable	Cumple	No cumple	Observaciones
1. Diseño y fundamentos REST			
Los endpoints del dominio están definidos con criterio REST (recurso – verbo HTTP – ruta – código de respuesta).			
Implementa parámetros de ruta y de consulta con validación (Query, Path, Annotated / type hints).			
El CRUD completo (crear, listar, consultar, actualizar, eliminar) está implementado sobre los recursos principales del dominio.			
2. Modelado y validación de datos (Pydantic)			
Usa esquemas Pydantic v2 separados para entrada y salida (Create / Update / Response).			
Incluye validaciones propias del dominio (field_validator / model_validator, restricciones de Field).			
3. Persistencia de datos (SQLAlchemy)			
Define al menos dos entidades relacionadas mediante modelos SQLAlchemy 2.0.			
El CRUD es persistente contra una base de datos real (SQLite / PostgreSQL), no en memoria.			
4. Autenticación y autorización			
Implementa inicio de sesión con JWT (flujo OAuth2 password u equivalente).			
Protege endpoints sensibles según autenticación y/o rol del usuario.			
5. Manejo de errores y middlewares			
Maneja errores de forma estandarizada (HTTPException, códigos 404/422, mensajes claros).			
Configura CORS para permitir el consumo de la API desde el frontend en React.			
6. Asincronía y tareas en segundo plano			
Incluye al menos un endpoint o flujo implementado con async/await.			
Usa BackgroundTasks (u otro mecanismo equivalente) para al menos una tarea no bloqueante.			
7. Integración de Inteligencia Artificial			
El proyecto expone al menos un endpoint que integra un modelo propio o un servicio de IA externo.			

Criterio verificable	Cumple	No cumple	Observaciones
Las credenciales o llaves de API se gestionan por variables de entorno, nunca en el código.			
8. Documentación y preparación para despliegue			
La documentación automática (/docs, /redoc) está personalizada con tags, descripciones y ejemplos.			
Incluye README con instrucciones de instalación y ejecución, y requirements.txt / .env.example.			
9. Pruebas (Testing)			
Incluye pruebas con Pytest / TestClient que cubren al menos el CRUD y la autenticación.			
10. Comparativa técnica y sustentación			
Incluye un análisis o sección comparativa entre FastAPI y Django REST Framework aplicado al proyecto.			
El aprendizaje sustenta con claridad el funcionamiento del proyecto y las decisiones técnicas tomadas.			
11. Manual técnico			
Entrega un manual técnico del proyecto: arquitectura, requisitos, instalación, configuración y operación.			
12. Integración y despliegue continuo (CI/CD)			
Existe un flujo de integración continua en GitHub que revisa el estilo y ejecuta las pruebas automáticamente en cada cambio.			
El despliegue a la nube es automático y está condicionado a que la integración continua termine correctamente.			
13. Pasarela de pago			
El proyecto implementa un flujo de pago completo con validación del medio de pago y emisión del comprobante.			
Los datos sensibles del medio de pago no se almacenan (no se guarda el número completo ni el código de seguridad).			
14. Base de datos (normalización)			
El modelo de datos está normalizado hasta la tercera forma normal (1FN, 2FN y 3FN), y las desnormalizaciones están justificadas.			
Las relaciones declaran integridad referencial y restricciones de unicidad coherentes con el dominio.			
15. Respuesta a conceptos y principios			
Explica con ejemplos del propio proyecto qué son clase, objeto, herencia y polimorfismo.			
Explica en qué proceso las clases se convierten en objetos (instanciación: __new__ y el constructor __init__).			
Explica la utilidad de cada uno de los componentes del proyecto (capas del backend y carpetas del frontend).			
16. Seguridad: prevención de inyección SQL			
FORMA 1 — Frontend: el inicio de sesión está escalonado en dos formularios, de modo que la contraseña no forma parte de ninguna consulta.			
FORMA 2 — Backend: las consultas son preparadas (prepared statement): el valor viaja como parámetro y no dentro de la instrucción SQL.			
Total de criterios evaluados	32	Criterios que cumple	_____

Resultado general	☐ Aprueba ☐ No aprueba ☐ Aprueba con observaciones
--------------------------	---

Observaciones generales:

César Augusto Moreno Mena
Instructor técnico

JUAN DIEGO CARTAGENA TUBERQUIA
Aprendiz